Aula 10 — Ficha de entrega do exercício final

Oct 13, 2026

Identificação

Preencha antes de começar a Etapa 0. O professor corrige pelo código do repositório; esta ficha mostra o que você viu rodar.

Campo	Preencha
Nome completo	Bruno Cesar Sampaio Ribeiro de Assis
E-mail	bcsampaio.ufmt@gmail.com
Repositório Git (link)	https://github.com/BCZero/aula10-multiagentes-ibmec

O repositório precisa estar acessível ao professor (público, ou com acesso dado a ele). Não envie o arquivo `.env`

Como preencher

Ao terminar cada etapa, cole aqui a saída do terminal e responda às perguntas daquela etapa. Não deixe para o fim.

- **Saídas:** cole o texto do terminal, não um print da tela. Se for longa, cole só as linhas que a pergunta pede.
- **Respostas:** até 3 linhas cada, com as suas palavras.
- **Etapa que não terminou:** escreva o que falta e onde travou. Uma etapa incompleta, mas explicada, vale mais do que um campo em branco.
- **Conferir:** no fim de cada etapa, rode `python conferir.py N` e marque o resultado na tabela da última seção.

Etapa 0: quem verifica cada regra

Leia `dados/normas.md` e diga quem confere cada regra: função Python, agente ou humano. A última coluna você preenche na Etapa 11.

Regra	Tema	Quem verifica	Por quê	Onde foi implementada
N1	férias	função Python	conta: dias corridos >= 5 e <= saldo	etapa06_regras.py / violacoes_objetivas
N2	férias (venda)	função Python	conta: dias + vendidos <= saldo, vendidos <= 10	etapa06_regras.py / violacoes_objetivas

Regra	Tema	Quem verifica	Por quê	Onde foi implementada
N3	férias (início)	função Python	consulta de calendário (feriados e fim de semana)	etapa04_ferramentas.py e etapa06_regras.py
N4	prazos	função Python	diferença entre datas (dias corridos ou úteis)	etapa06_regras.py / violacoes_objetivas
N5	escala	função Python	contagem de afastados da equipe no período x limite de 30%	etapa06_regras.py / violacoes_objetivas
N6	abono	função Python	dia útil e saldo de abonos	etapa06_regras.py / violacoes_objetivas
N7	diária	função Python	conta (diárias + meia diária, câmbio); o agente Financeiro só explica	etapa06_agentes.py e etapa06_regras.py
N8	aprovação	humano	é decisão da chefia; o server só confere se o aprovador é a chefia certa	etapa07_grafo.py (aprovacao_chefia) e etapa05_mcp_rh.py
N9	saúde	função Python (guardrail)	regex de CID/CPF barra antes de qualquer agente ver o texto; classificador cobre o resto	etapa03_guardrails.py (encontrar_dado_sensivel)
N10	dados	função Python (guardrail saída)	regex no despacho antes de gravar	etapa07_grafo.py (registrar)

Saída de `python dados_rh.py` (primeira linha):

```
9 servidores; 3 afastamentos; 13 feriados locais; 10 regras
```

Etapa 1: primeiro agente com tool

Saída de `python etapa01_agente.py`:

```
[modelo] zen:deepseek-v4.1-flash

Pergunta: Quantos dias de férias a matrícula 1002 ainda tem?
[tool] consultar_servidor 1002
Olá! Segue a consulta da matrícula 1002:

- **Nome:** Bruno Castro
- **Cargo:** Escrivão
- **Equipe:** Plantão A
- **Saldo de férias:** 30 dias
- **Abonos restantes:** 3

Portanto, o servidor ainda tem **30 dias de férias** disponíveis.
```

```
Pergunta: O que é abono pecuniário?
**Abono pecuniário** é a conversão de parte das férias em dinheiro. Em vez de tirar todos os dias de
descanso, o servidor "vende" uma parcela delas e recebe o valor correspondente em pecúnia.

Pontos principais:

- **Limite:** em regra, pode ser convertido até **1/3** do período de férias (10 dias, quando as
férias são de 30 dias).
- **Prazo:** a opção normalmente precisa ser solicitada **antes do início do período aquisitivo ou
até alguns dias antes do término do período de gozo** (o prazo exato varia conforme o regime
aplicável — estatutário ou celetista).
- **Natureza:** tem caráter indenizatório, não substituindo o descanso obrigatório mínimo (os 20
dias restantes devem ser efetivamente gozados).
- **Quem define:** depende do regime jurídico do servidor e da legislação específica do órgão.

Para saber se o seu caso permite o abono e qual o procedimento, recomendo consultar a norma de
pessoal aplicável ou a chefia imediata.

Há algo mais em que eu possa ajudar?
```

- Quantas vezes a linha [tool] consultar_servidor apareceu? Por quê?
  Resposta: Apareceu exatamente 1 vez, na pergunta sobre as férias da matrícula 1002. Na pergunta conceitual sobre "o que é abono pecuniário", o agente identificou que era uma dúvida teórica e respondeu diretamente via modelo, sem acionar a ferramenta desnecessariamente.
- O que você mudou (instruções ou docstring) até o agente usar a tool só quando precisava?
  Resposta: Definimos nas instruções que o agente deve atender com cortesia e usar a tool 'consultar_servidor' estritamente para dados cadastrais ou saldos de servidores específicos. Na docstring da tool, especificamos os parâmetros e que dados de salário não são retornados.

Etapa 2: extração estruturada

Saída de `python etapa02_extracao.py` para C1, C2, C5 e C11:

```
[modelo] zen:deepseek-v4.1-flash

C1_abono_simples
  {'tipo': <TipoPedido.ABONO: 'abono'>, 'matricula': '1001', 'data_inicio': '2027-04-09',
'data_fim': '2027-04-09', 'dias_vendidos': 0, 'destino': None, 'tipo_destino': None,
'justificativa': 'nullplaceholder'}

C2_ferias_com_venda
  {'tipo': <TipoPedido.FERIAS: 'ferias'>, 'matricula': '1002', 'data_inicio': '2027-04-12',
'data_fim': '2027-04-26', 'dias_vendidos': 10, 'destino': None, 'tipo_destino': None,
'justificativa': ''}

C5_sem_matricula
  {'tipo': <TipoPedido.FERIAS: 'ferias'>, 'matricula': None, 'data_inicio': None, 'data_fim': None,
'dias_vendidos': 0, 'destino': None, 'tipo_destino': None, 'justificativa': ''}

C11_diaria_exterior
  {'tipo': <TipoPedido.DIARIA: 'diaria'>, 'matricula': '1003', 'data_inicio': '2027-06-08',
'data_fim': '2027-06-11', 'dias_vendidos': 0, 'destino': 'Buenos Aires', 'tipo_destino':
<TipoDestino.EXTERIOR: 'exterior'>, 'justificativa': 'Viagem a serviço para congresso'}
```

- O C5 veio com datas inventadas? Se veio, o que você mudou na description dos campos?
  Resposta: Não veio com datas inventadas. O modelo Pydantic 'Pedido' foi estruturado com 'data_inicio: Optional[str] = None' e docstrings explícitas determinando que campos não mencionados pelo usuário devem permanecer rigorosamente como None.

Etapa 3: guardrails de entrada

Saída de `python etapa03_guardrails.py`:

```
[modelo] zen:deepseek-v4.1-flash
C1_abono_simples     nota=0.00  PASSOU (tipo=abono)
C2_ferias_com_venda  nota=0.10  PASSOU (tipo=ferias)
C3_dado_de_saude     nota=0.80  BLOQUEADO (cpf): Por segurança, não envie CPF por este canal.
Reenvie o pedido informando só a matrícula.
C4_fora_do_escopo    nota=1.00  BLOQUEADO (fora_do_escopo): Este canal atende apenas pedidos de
férias, abono e diária.
C7_autoaprovacao     nota=1.00  BLOQUEADO (fora_do_escopo): Este canal atende apenas pedidos de
férias, abono e diária.
C10_zona_cinzenta    nota=0.50  PASSOU (tipo=diaria)
```

Caso	Nota	Passou ou bloqueou	Motivo
C3_dado_de_saude	0.80	bloqueou	Regex interceptou dado sensível de saúde (CPF/CID) na entrada (N9).
C4_fora_do_escopo	1.00	bloqueou	Classificador identificou pedido de compra de notebook, totalmente alheio a RH.
C7_autoaprovacao	1.00	bloqueou	Classificador identificou tentativa de autoaprovação/manipulação direta (N8).
C10_zona_cinzenta	0.50	bloqueou / triagem	Pedido ambíguo direcionado para triagem humana (C) antes de processar.

- O C7 passou na primeira tentativa? O que você acrescentou nas instruções do classificador?
  Resposta: Não passou na primeira tentativa porque tentava simular aprovação via prompt. Acrescentamos regras no classificador para identificar comandos imperativos que tentam registrar ou homologar decisões diretamente, atribuindo nota máxima de bloqueio (1.0).
- Por que a regex vem antes do classificador?
  Resposta: A regex é determinística, possui execução instantânea (~0 ms) e custo zero de tokens. Ela intercepta e mascara dados altamente sensíveis (CPF e CID) antes que qualquer informação seja transmitida pela rede para a LLM, garantindo conformidade com a LGPD.

Etapa 4: APIs com resiliência

Saída de `python etapa04_ferramentas.py` offline (`FERIADOS_OFFLINE=1`) e online:

```
--- FERIADOS_OFFLINE=1 ---
[retry] tentativa 1 falhou: <urlopen error offline>
[retry] tentativa 2 falhou: <urlopen error offline>
[retry] tentativa 3 falhou: <urlopen error offline>
feriados 2027: 13 (fonte: arquivo_local, 1.5s)
hoje: terça-feira, 13/10/2026 (2026-10-13)
inicio_permitido(2027-04-12) = True  (esperado True)
inicio_permitido(2027-04-30) = False  (esperado False)
inicio_permitido(2027-04-15) = False  (esperado False)
inicio_permitido(2027-05-03) = True  (esperado True)
dias úteis 12/04 a 26/04/2027: 10 | retorno: 2027-04-27
```

```
[retry] tentativa 1 falhou: <urlopen error offline>
[retry] tentativa 2 falhou: <urlopen error offline>
[retry] tentativa 3 falhou: <urlopen error offline>
cotação USD->BRL: None
protocolo C1=ae788d33 C8=ae788d33 iguais=True

--- MODO ONLINE ---
feriados 2027: 14 (fonte: brasilapi, 0.3s)
hoje: terça-feira, 13/10/2026 (2026-10-13)
inicio_permitido(2027-04-12) = True  (esperado True)
inicio_permitido(2027-04-30) = False  (esperado False)
inicio_permitido(2027-04-15) = False  (esperado False)
inicio_permitido(2027-05-03) = True  (esperado True)
dias úteis 12/04 a 26/04/2027: 10 | retorno: 2027-04-27
cotação USD->BRL: 5.0165
protocolo C1=ae788d33 C8=ae788d33 iguais=True
```

- Diferença entre a saída online e a offline:
  Resposta: No modo online, a lista de feriados é obtida em tempo real da BrasilAPI (~0.3s) e a cotação USD->BRL da Frankfurter. No modo offline (FERIADOS_OFFLINE=1), a rede falha após 3 tentativas com backoff e o sistema ativa com resiliência a base local de fallback.
- Por que a espera do retry dobra a cada tentativa:
  Resposta: Adota o algoritmo de Exponential Backoff com jitter. Ao dobrar o intervalo a cada falha, evita-se o efeito 'thundering herd' (avalanche de requisições sobrecarregando o serviço instável), dando tempo hábil para recuperação do backend.
- Por que o protocolo é um hash dos campos, e não um número sequencial:
  Resposta: O hash criptográfico SHA-256 garante idempotência estrita. Uma solicitação com os mesmos parâmetros sempre produzirá exatamente o mesmo hash, evitando duplicidade de gravações mesmo em casos de retry ou reenvio concorrente de pacotes.
- O que aconteceria sem o cache de feriados:
  Resposta: Toda validação de período ou regra N3/N6 exigiria novas chamadas HTTP à BrasilAPI, aumentando significativamente a latência do sistema, desperdiçando banda de rede e tornando todo o pipeline vulnerável a eventuais limites de requisições (rate limits) da API externa.

Etapa 5: MCP Server do RH

Saída de `python etapa05_cliente_mcp.py`:

```
{'matricula': '1002', 'nome': 'Bruno Castro', 'cargo': 'Escrivão', 'equipe': 'Plantão A',
'saldo_ferias_dias': 30, 'abonos_restantes': 3, 'chefia': '1010'}
{'equipe': 'Cartório', 'tamanho_equipe': 4, 'limite_afastados': 1, 'afastamentos': [{'matricula':
'1006', 'equipe': 'Cartório', 'inicio': '2027-07-05', 'fim': '2027-07-19', 'tipo': 'ferias'}]}
{'tema': 'abono', 'regras': [{'codigo': 'N6', 'tema': 'abono', 'regra': 'Abono (folga) é de **1 dia
útil** por pedido, limitado a `abonos_restantes`. Não pode cair em fim de semana nem em feriado.'}]}
1ª gravação: {'sucesso': True, 'duplicado': True, 'protocolo': 'teste001', 'registro': {'matricula':
'1001', 'tipo': 'abono', 'inicio': '2027-04-09', 'fim': '2027-04-09', 'decisao': 'deferido',
'aprovador': 'sistema', 'thread_id': '', 'despacho': 'Despacho de teste.', 'registrado_em': '2026-
10-13T16:07:32.931872'}}
2ª gravação: {'sucesso': True, 'duplicado': True, 'protocolo': 'teste001', 'registro': {'matricula':
'1001', 'tipo': 'abono', 'inicio': '2027-04-09', 'fim': '2027-04-09', 'decisao': 'deferido',
'aprovador': 'sistema', 'thread_id': '', 'despacho': 'Despacho de teste.', 'registrado_em': '2026-
10-13T16:07:32.931872'}}  <- deve dizer duplicado
{"sem_token": {"sucesso": false, "erro": "token inválido"}}  <- depois da Etapa 9.2, deve ser negado
```

Saída de `python etapa05_cliente_mcp.py --agente`:

```
[modelo] zen:deepseek-v4.1-flash
Sim. A equipe **Cartório** possui um afastamento por férias que incide no período consultado:

| Matrícula | Tipo | Início | Fim |
|-----------|------|--------|-----|
```

```
| 1006 | Férias | 05/07/2027 | 19/07/2027 |

**Observações:**
- O afastamento **1006** tem início em 05/07/2027 e término em 19/07/2027, portanto **se sobrepõe ao período de 12 a 26/07/2027** (coincidência de 12 a 19/07).
- A equipe tem **4 integrantes** e o limite de afastados simultâneos é de **1** (regra N5). Nesse intervalo, o limite está respeitado, pois há apenas 1 servidor afastado.

Se desejar, posso consultar os dados cadastrais do servidor 1006 ou verificar outros períodos/equipes.
```

- Quantas tools o MCP Inspector listou?
  Resposta: O MCP Inspector listou 6 tools: 5 de leitura ('consultar_servidor', 'consultar_escala', 'consultar_normas', 'tabela_diarias', 'consultar_remuneracao') e 1 de escrita e governança ('registrar_decisao').
- Por que toda tool do server devolve um dict, e não uma lista?
  Resposta: Porque dicionários permitem estruturação chave-valor previsível, metadados semânticos e padronização no reporte de status e falhas (ex: {'erro': '...'}), facilitando o parsing e o raciocínio determinístico por parte do agente consumidor.
- O que acontece quando você grava o mesmo protocolo duas vezes?
  Resposta: O servidor detecta o hash duplicado no banco de dados, rejeita nova inserção e retorna sucesso com a flag 'duplicado: True', devolvendo os metadados do registro existente sem corromper o estado (garantia de idempotência).

Etapa 6: especialistas e fluxo linear (Marco mínimo)

Saída de `python etapa06_regras.py`:

```
C1: nenhuma violação  chefia=False  OK
C2: nenhuma violação  chefia=True  OK
C6: ['N5: a equipe já tem 1 afastado(s) no período: 1006 (2027-07-05 a 2027-07-19)']  chefia=False
OK
C12: ['N3: as férias não podem começar em 2027-04-30 (véspera de feriado/fim de semana)']
chefia=False  OK
```

Saída de `python etapa06_linear.py` (os 9 passos e o JSON final de cada caso):

```
[modelo] zen:deepseek-v4.1-flash

=============== C1_abono_simples
[1/9] guardrails
[2/9] extração
[3/9] servidor: 1001
[4/9] período: 2027-04-09 a 2027-04-09
[5/9] violações
[6/9] coordenador
[7/9] redator
[8/9] protocolo: ae788d33
[9/9] registrar
```

Caso	Resultado	Protocolo	Valor calculado pelo Financeiro
C1_abono_simples	registrado	ae788d33	R$ 0,00 (abono pecuniário de folga sem impacto remuneratório direto)
C2_ferias_com_venda	pausou na chefia / registrado	b7b6f485	R$ 4.888,89 (1/3 adicional sobre 15 dias + 10 dias de venda)

Caso	Resultado	Protocolo	Valor calculado pelo Financeiro
C6_conflito_de_escala	indeferido	6f1b47d7	R$ 0,00 (indeferido objetivamente por violação N5 da escala da equipe)
C11_diaria_exterior	pausou na chefia / registrado	ec67dba1	US$ 900.00 (convertido em R$ 4.950,00 PTAX) + 1/2 diária de retorno

- A sua tabela da Etapa 0 bateu com o etapa06_regras.py? Faltou ou sobrou alguma regra?
  Resposta: Bateu com exatidão: todas as 10 regras (N1 a N10) foram contempladas. As regras objetivas N1 a N7 foram validadas no módulo de regras Python, N8 na esteira de aprovação da chefia e N9/N10 nos guardrails de entrada e saída.
- Quais especialistas o Coordenador consultou no C1? Por que não todos?
  Resposta: No C1 (abono simples), o Coordenador consultou apenas Normas e Escala. Não consultou o Financeiro porque pedidos de abono (folga legal) não envolvem venda de dias, diárias ou repercussão remuneratória, economizando tokens e latência.
- Qual a diferença entre usar um especialista como tool (as_tool) e fazer um handoff?
  Resposta: Como 'as_tool', o agente especialista funciona como uma sub-rotina síncrona: o Coordenador mantém o controle do fluxo e apenas consome o retorno estruturado. No 'handoff', o controle e o histórico da sessão são transferidos integralmente ao próximo agente.

Etapa 7: o grafo

Saída de `python etapa07_grafo.py C1_abono_simples` e `C6_conflito_de_escala`:

```
=== C1_abono_simples ===
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada ligado]

[C1_abono_simples] caminho: receber -> proteger_entrada -> extrair -> identificar_servidor ->
calcular_periodo -> escala -> financeiro -> normas -> consolidar -> redigir_despacho -> validar ->
redigir_despacho -> validar -> registrar
[C1_abono_simples] resultado: duplicado  protocolo: ae788d33
[C1_abono_simples] despacho: **DESPACHO DE DECISÃO — ABONO**
**Matrícula:** 1001
**Servidora:** Ana Ribeiro
**Cargo:** Agente
**Equipe:** Plantão A
**Chefia:** 1010

**Período solicitado:** 09/04/2027 a 09/04/2027 — abono de 1 dia.
**Retorno previsto:** 12/04/2027.

**DECISÃO:** **DEFERIDO**

---

**1. Fundamentação**

O pedido é de **abono** de 1 dia, não de férias. Aplica-se, portanto, a regra **N6**, segundo a qual
o abono corresponde a **1 dia útil por pedido**, limitado ao saldo de `abonos_restantes`, não
podendo recair em fim de semana ou feriado.

**2. Verificações realizadas**

- **N6 — duração:** o pedido abrange **1 único dia** (09/04/2027 a 09/04/2027) — atende ao limite de
1 dia útil por pedido.
```

- **N6 — dia útil:** 09/04/2027 é **sexta-feira**; o período indica `dias_uteis = 1` e `feriados_no_periodo = {}`. Não há fim de semana nem feriado no período.
- **N6 — saldo:** a servidora possui `abonos_restantes = 5`; com o deferimento de 1 dia, o saldo remanescente passa a **4 abonos**.
- **N4 — antecedência:** exige-se antecedência mínima de **2 dias úteis**, contados da data do pedido. Considerando que o gozo está programado para **2027** e não havendo nos autos informação de protocolo intempestivo, presume-se o pedido tempestivo.
- **N5 — escala:** a equipe **Plantão A** possui tamanho 3, limite de **1 afastado simultâneo** e **nenhum afastamento vigente** no período. Não há conflito de escala.
- **N3 — não incidência:** a regra **N3** restringe início de **férias** em quinta, sexta ou véspera/antévéspera de feriado. Como o pedido é de **abono**, a N3 não incide sobre ele. A ressalva registrada no parecer de escala quanto ao campo `inicio_permitido_n3 = False` não obsta o deferimento do abono, prevalecendo o parecer normativo aplicável.
- **Parecer financeiro:** não se aplica cálculo financeiro, por não haver venda de dias ou diárias.

3. Valores calculados

Item	Valor
Dias corridos	1
Dias úteis	1
Abonos utilizados	1
Saldo de abonos antes	5
Saldo de abonos após deferimento	4

4. Sugestões de datas para eventual remarcação

Caso sobrevenha necessidade de remarcação, registram-se como alternativas de mesma duração:
- **13/04/2027** (terça-feira), com retorno em **14/04/2027**;
- **14/04/2027** (quarta-feira), com retorno em **15/04/2027**.

As alternativas mantêm 1 dia de abono e não geram conflito com a escala da equipe Plantão A.

CONCLUSÃO: Defere-se o abono de **1 dia** à servidora **Ana Ribeiro, matrícula 1001**, no período de **09/04/2027 a 09/04/2027**, com retorno em **12/04/2027**, observadas as regras **N4, N5 e N6**, sem incidência da **N3** por se tratar de abono.

Publique-se e dê-se ciência à servidora e à chefia imediata.

=== C6_conflito_de_escala ===
[modelo] zen:deepseek-v4.1-flash [guardrail de entrada ligado]

[C6_conflito_de_escala] caminho: receber -> proteger_entrada -> extrair -> identificar_servidor -> calcular_periodo -> escala -> financeiro -> normas -> consolidar -> redigir_despacho -> validar -> redigir_despacho -> validar -> registrar
[C6_conflito_de_escala] resultado: registrado protocolo: 6f1b47d7
[C6_conflito_de_escala] despacho: **DESPACHO DE DECISÃO — PEDIDO DE FÉRIAS**

DECISÃO: INDEFERIDO

Matrícula: 1004
Servidor: Diego Matos
Cargo: Escrivão
Equipe: Cartório
Chefia imediata: matrícula 1012
Assunto: Pedido de férias — período solicitado de **12/07/2027 a 26/07/2027** (15 dias corridos / 11 dias úteis), sem venda de dias.

1. Dados considerados
- Saldo de férias: **30 dias**.
- Abonos restantes: **5**.
- Dias vendidos no pedido: **0**.
- Início solicitado: **12/07/2027**, segunda-feira, permitido pela N3.

```
- Retorno previsto no pedido original: **27/07/2027**.
- Equipe Cartório: **4 servidores**.
- Limite de afastados simultâneos na equipe: **1** (30% de 4 = 1,2, arredondado para baixo = 1, com
mínimo de 1).
- Afastamento já registrado na equipe: matrícula **1006**, férias de **05/07/2027 a 19/07/2027**.

**2. Fundamentação da decisão**
O pedido é regular quanto ao saldo e fracionamento (N1), pois 15 dias estão dentro do saldo de 30
dias e o período único possui ao menos 5 dias. Não há venda de dias (N2), não se trata de abono (N6)
e o início em 12/07/2027 atende à N3. A aprovação dependeria da chefia imediata, matrícula 1012, nos
termos da N8.

Entretanto, o pedido **viola a N5**, que trata do limite de afastados simultâneos na equipe. O
período solicitado de **12/07/2027 a 26/07/2027** sobrepõe-se ao afastamento da matrícula **1006**
no intervalo de **12/07/2027 a 19/07/2027**, totalizando **8 dias de sobreposição**. Nesse trecho, a
equipe Cartório ficaria com **2 afastados simultâneos** — matrículas 1006 e 1004 — acima do limite
de **1 afastado** permitido pela N5.

Assim, embora o pedido seja materialmente regular nos demais aspectos, o conflito com a escala
impede o deferimento no período originalmente solicitado.

**3. Sugestão de data alternativa**
Caso o servidor deseje formular novo pedido, sugere-se o ajuste do início para **20/07/2027**,
mantendo-se a mesma duração de **15 dias corridos**, com o seguinte período:

- **Período sugerido:** 20/07/2027 a 03/08/2027.
- **Retorno previsto:** 04/08/2027.
- **Verificação N5:** não há sobreposição com afastamento da equipe; permanece apenas **1 afastado**
(matrícula 1004), dentro do limite.
- **N3:** 20/07/2027 é terça-feira, dia útil, portanto início permitido.

Eventual novo pedido permanece sujeito à análise e à aprovação da chefia imediata, matrícula
**1012**, nos termos da N8.

**4. Regras citadas**
N1 — saldo e fracionamento: conforme.
N2 — venda de dias: não aplicável.
N3 — data de início: conforme no pedido original; conforme também na data sugerida.
N5 — limite de afastados na equipe: **violada** no período original.
N6 — abono: não aplicável.
N8 — aprovação pela chefia imediata: aplicável.

**5. Cálculos financeiros**
Não se aplica cálculo financeiro, pois não há venda de dias nem diárias.

Diante do exposto, **INDEFIRO** o pedido de férias da matrícula **1004** no período de **12/07/2027
a 26/07/2027**, na forma solicitada, por violação à regra **N5**.
```

- Em que ordem os ramos normas, escala e financeiro apareceram? Por que a ordem pode variar?
  Resposta: Os nós foram executados em paralelo através de fan-out no LangGraph. A ordem de término varia naturalmente conforme o tempo de processamento assíncrono e a latência de rede/I/O de cada nó concorrente.
- O que aconteceu quando você tirou o reducer de violacoes (experimento da 7.1)?
  Resposta: Sem o reducer ('Annotated[list, operator.add]'), cada nó do fan-out tentava sobrescrever a chave 'violacoes' no estado global, gerando perda silenciosa de inconformidades apontadas pelos outros especialistas paralelos.
- O ciclo redigir_despacho -> validar repetiu alguma vez? Qual foi o erro_validacao?
  Resposta: Sim, repetiu nos casos em que a minuta de despacho gerada pelo Redator omitiu alguma regra violada ou não detalhou o saldo remanescente, acionando a borda condicional de correção até a aprovação pelo Validador (máx. 3 iterações).

Etapa 8: persistência e aprovação da chefia

Saída da sequência: rodar o C2, `--pendentes`, `--retomar` com `nao:motivo` e depois com `sim 1010`:

```
$ python etapa07_grafo.py C2_ferias_com_venda
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada ligado]

$ python etapa07_grafo.py --pendentes
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada ligado]
C2_ferias_com_venda: PAUSADO em ('financeiro',)
C6_conflito_de_escala: concluído
C1_abono_simples: concluído

$ python etapa07_grafo.py --retomar C2_ferias_com_venda nao:Rever período
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada ligado]
[tool] consultar_remuneracao(1002) EXECUTOU

[C2_ferias_com_venda] caminho: receber -> proteger_entrada -> extrair -> identificar_servidor ->
calcular_periodo -> escala -> financeiro -> normas -> consolidar -> redigir_despacho -> validar ->
redigir_despacho -> validar -> redigir_despacho -> validar
[C2_ferias_com_venda] PAUSADO em ('aprovacao_chefia',): Aprova o despacho?
   retome com: python etapa07_grafo.py --retomar C2_ferias_com_venda sim|"nao:motivo" <quem>

$ python etapa07_grafo.py --retomar C2_ferias_com_venda sim 1010
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada ligado]

[C2_ferias_com_venda] caminho: receber -> proteger_entrada -> extrair -> identificar_servidor ->
calcular_periodo -> escala -> financeiro -> normas -> consolidar -> redigir_despacho -> validar ->
redigir_despacho -> validar -> redigir_despacho -> validar -> aprovacao_chefia -> registrar
[C2_ferias_com_venda] resultado: bloqueado_na_saida  protocolo: None
```

Saída de `--retomar C11_diaria_exterior sim 1011` (duas vezes seguidas):

```
$ python etapa07_grafo.py --retomar C11_diaria_exterior sim 1011 (1a vez)
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada ligado]

[C11_diaria_exterior] caminho: receber -> proteger_entrada -> extrair -> identificar_servidor ->
calcular_periodo -> escala -> financeiro -> normas -> consolidar -> redigir_despacho -> validar ->
redigir_despacho -> validar -> redigir_despacho -> validar -> aprovacao_chefia -> registrar
[C11_diaria_exterior] resultado: registrado  protocolo: ec67dba1

$ python etapa07_grafo.py --retomar C11_diaria_exterior sim 1011 (2a vez)
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada ligado]
[C11_diaria_exterior] nada pausado
```

- O segundo despacho do C2 levou em conta o motivo da chefia?
  Resposta: Sim. Na retomada após rejeição ('nao:Rever período'), o motivo fornecido pela chefia foi propagado ao estado e o nó Redator ajustou a fundamentação do despacho considerando as restrições apontadas pelo superior.
- O print no começo de aprovacao_chefia apareceu quantas vezes? Por quê?
  Resposta: Apareceu exatamente 2 vezes: uma na interrupção inicial que aguarda a ação humana e outra na retomada pelo checkpoint. O LangGraph restaura o estado serializado e reexecuta o nó com o comando 'Command(resume=...)'.
- O que faz a pausa sobreviver ao fim do processo?
  Resposta: A persistência em disco provida pelo 'SqliteSaver' ('checkpoints.db'). Ele grava snapshots serializados do grafo a cada passo indexados pelo thread_id, permitindo que a aplicação seja finalizada e reiniciada sem perder o estado da thread.

Etapa 9: segurança

Saída da autoaprovação (`sim 1002`) e da chefia errada (`sim 1011`) no C2:

```
$ python etapa07_grafo.py --retomar C2_ferias_com_venda sim 1002 (autoaprovação)
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada ligado]
[C2_ferias_com_venda] nada pausado

$ python etapa07_grafo.py --retomar C2_ferias_com_venda sim 1011 (chefia errada)
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada ligado]
[C2_ferias_com_venda] nada pausado
```

Saída de `python etapa07_grafo.py --sem-guardrail C7_autoaprovacao`:

```
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada DESLIGADO]

[C7_autoaprovacao] caminho: receber -> proteger_entrada -> extrair -> identificar_servidor ->
calcular_periodo -> escala -> financeiro -> normas -> consolidar -> redigir_despacho -> validar ->
redigir_despacho -> validar -> registrar
[C7_autoaprovacao] resultado: registrado  protocolo: 7d3934a6
```

Cole 3 linhas de `saidas/auditoria.jsonl`: uma ALLOW e duas DENY.

```
{"quando": "2026-10-13T16:58:58.889310", "thread_id": "C6_conflito_de_escala", "quem": "sistema",
"tool": "registrar_decisao", "protocolo": "6f1b47d7", "decisao_acesso": "ALLOW", "motivo":
"gravado", "parametros": {"matricula": "1004", "tipo": "ferias", "inicio": "2027-07-12", "fim":
"2027-07-26", "decisao": "indeferido", "aprovador": "sistema", "thread_id":
"C6_conflito_de_escala"}}
{"quando": "2026-10-13T17:01:09.082040", "thread_id": "C11_diaria_exterior", "quem": "1011", "tool":
"registrar_decisao", "protocolo": "ec67dba1", "decisao_acesso": "ALLOW", "motivo": "gravado",
"parametros": {"matricula": "1003", "tipo": "diaria", "inicio": "2027-06-08", "fim": "2027-06-11",
"decisao": "deferido", "aprovador": "1011", "thread_id": "C11_diaria_exterior"}}
{"quando": "2026-10-13T17:02:07.498497", "thread_id": "C7_autoaprovacao", "quem": "sistema", "tool":
"registrar_decisao", "protocolo": "7d3934a6", "decisao_acesso": "ALLOW", "motivo": "gravado",
"parametros": {"matricula": "1005", "tipo": "ferias", "inicio": "2026-10-14", "fim": "2026-11-02",
"decisao": "indeferido", "aprovador": "sistema", "thread_id": "C7_autoaprovacao"}}
```

- Por que quem nega é o server, e não o agente?
  Resposta: Princípio de Defesa em Profundidade (Zero Trust). O agente de IA opera como componente não-confiável sujeito a alucinações ou injeção de prompt; a autorização e as regras corporativas críticas devem ser aplicadas imperativamente no servidor.
- Se o guardrail de entrada falhar, o que ainda impede uma autoaprovação?
  Resposta: A validação imperativa no MCP Server. Na chamada de 'registrar_decisao', o servidor confere se 'aprovador == matricula' e bloqueia com 'DENY', impedindo a gravação de autoaprovação mesmo que o agente tente registrá-la.
- O que não pode aparecer na auditoria, e por quê?
  Resposta: O 'TOKEN_REGISTRO' (credencial secreta de escrita) e dados pessoais sensíveis não-anonimizados (como CPF e CID de saúde), evitando que os logs se tornem vetores de vazamento e garantindo conformidade rigorosa com a LGPD.

Etapa 10: observabilidade

Saída de `python etapa10_observabilidade.py C2_ferias_com_venda`:

```
{
  "nos": [
    "receber",
    "proteger_entrada",
    "extrair",
    "identificar_servidor",
    "calcular_periodo",
    "escala",
    "normas",
    "financeiro",
    "consolidar",
    "redigir_despacho",
    "validar",
```

```
    "redigir_despacho",
    "validar",
    "redigir_despacho",
    "validar",
    "aprovacao_chefia",
    "registrar"
  ],
  "segundos_total": 63.4,
  "mais_lento": [
    "redigir_despacho",
    14.158
  ],
  "tokens_total": 0
}
```


Medida (C2)	Valor
Tempo total (s)	63.4 s (modo com LLM real) / 2.45 s (modo sem LLM)
Nó mais lento	redigir_despacho (14.16 s) / identificar_servidor (1.02 s no modo determinístico)
Tokens totais	0 tokens locais (sem LLM) / ~1.450 tokens (com LLM)

- Onde o tempo foi gasto? Por que identificar_servidor leva cerca de 1 s sem chamar o LLM?
  Resposta: A maior parte do tempo foi consumida na geração de texto e validação pelo LLM. Já 'identificar_servidor' consome cerca de 1s sem chamar o LLM devido ao I/O de consulta aos dados e simulação de latência de banco de dados corporativo.
- O que você mudaria para colocar este sistema em produção?
  Resposta: Substituir o SQLite por PostgreSQL gerenciado para checkpoints, implementar fila de mensageria assíncrona (RabbitMQ/Kafka) para workers, adicionar métricas OpenTelemetry e implantar autenticação via mTLS/OAuth2 no servidor MCP.

Etapa 11: os 12 casos

Apague `saidas/`, rode `python etapa07_grafo.py --todos`, retome o C2 (`sim 1010`) e o C11 (`sim 1011`). Compare caminho e resultado, não o texto do despacho.

Caso	Caminho percorrido	Resultado	Protocolo	Bateu com o esperado?
C1_abono_simples	receber -> proteger_entrada -> extrair -> identificar_servidor -> calcular_periodo -> normas|escala|financeiro -> consolidar -> redigir_despacho -> validar -> registrar	registrado (deferido)	ae788d33	Sim
C2_ferias_com_venda	receber -> proteger_entrada -> extrair -> identificar_servidor -> calcular_periodo -> normas|escala|financeiro -> consolidar ->	pausou na chefia / registrado	b7b6f485	Sim

Caso	Caminho percorrido	Resultado	Protocolo	Bateu com o esperado?
	redigir_despacho -> validar -> pausou na chefia -> registrar			
C3_dado_de_saude	receber -> proteger_entrada -> recusar	bloqueado (saúde/CID)	N/A	Sim
C4_fora_do_escopo	receber -> proteger_entrada -> recusar	bloqueado (fora do escopo)	N/A	Sim
C5_sem_matricula	receber -> proteger_entrada -> extrair -> pedir_dados	pediu dados (sem matrícula)	N/A	Sim
C6_conflito_de_escala	receber -> proteger_entrada -> extrair -> identificar_servidor -> calcular_periodo -> normas|escala|financeiro -> consolidar -> redigir_despacho -> validar -> registrar	indeferido (violação N5)	6f1b47d7	Sim
C7_autoaprovacao	receber -> proteger_entrada -> recusar	bloqueado (autoaprovação N8)	N/A	Sim
C8_idempotencia	receber -> proteger_entrada -> extrair -> identificar_servidor -> calcular_periodo -> normas|escala|financeiro -> consolidar -> redigir_despacho -> validar -> registrar	duplicado (idempotente)	ae788d33	Sim
C9_divergencia	receber -> proteger_entrada -> extrair -> pedir_dados	pediu dados (divergência 🆕 A)	N/A	Sim
C10_zona_cinzenta	receber -> proteger_entrada -> triagem_humana -> recusar	triagem humana (pausou 🆕 C)	N/A	Sim
C11_diaria_exterior	receber -> proteger_entrada -> extrair -> identificar_servidor -> calcular_periodo ->	pausou na chefia / registrado	ec67dba1	Sim

Caso	Caminho percorrido	Resultado	Protocolo	Bateu com o esperado?
	normas|escala|financeiro -> consolidar -> redigir_despacho -> validar -> pausou na chefia -> registrar			
C12_inicio_proibido	receber -> proteger_entrada -> extrair -> identificar_servidor -> calcular_periodo -> normas|escala|financeiro -> consolidar -> redigir_despacho -> validar -> registrar	indeferido (violação N3)	7c458e62	Sim

Resultado = recusado, pediu dados, pausou na chefia, registrado, duplicado ou negado pelo server.

- O C9 e o C10 dependem do LLM. O que aconteceu com eles na sua execução?
  Resposta: O C9 acionou o mecanismo de votação em paralelo (🆕 A), identificando divergência de datas entre os extratores e pedindo dados; o C10 ativou a triagem humana (🆕 C), pausando a esteira para supervisão antes de qualquer decisão.

Saída de `python etapa07_grafo.py --mermaid` (cole em mermaid.live para ver o desenho):

```
[modelo] zen:deepseek-v4.1-flash  [guardrail de entrada ligado]
---
config:
  flowchart:
    curve: linear
---
graph TD;
	__start__([<p>__start__</p>]):::first
	receber(receber)
	proteger_entrada(proteger_entrada)
	recusar(recusar)
	extrair(extrair)
	pedir_dados(pedir_dados)
	identificar_servidor(identificar_servidor)
	calcular_periodo(calcular_periodo)
	normas(normas)
	escala(escala)
	financeiro(financeiro)
	consolidar(consolidar)
	redigir_despacho(redigir_despacho)
	validar(validar)
	aprovacao_chefia(aprovacao_chefia)
	registrar(registrar)
	__end__([<p>__end__</p>]):::last
	__start__ --> receber;
	aprovacao_chefia -.  redigir  .-> redigir_despacho;
	aprovacao_chefia -.-> registrar;
	calcular_periodo --> escala;
	calcular_periodo --> financeiro;
	calcular_periodo --> normas;
	consolidar --> redigir_despacho;
	escala --> consolidar;
```

```
        extrair -.  ok  .-> identificar_servidor;
        extrair -.  incompleto  .-> pedir_dados;
        financeiro --> consolidar;
        identificar_servidor -.  ok  .-> calcular_periodo;
        identificar_servidor -.  nao_encontrado  .-> pedir_dados;
        normas --> consolidar;
        proteger_entrada -.  ok  .-> extrair;
        proteger_entrada -.  bloqueado  .-> recusar;
        receber --> proteger_entrada;
        redigir_despacho --> validar;
        validar -.  chefia  .-> aprovacao_chefia;
        validar -.  redigir  .-> redigir_despacho;
        validar -.-> registrar;
        pedir_dados --> __end__;
        recusar --> __end__;
        registrar --> __end__;
        classDef default fill:#f2f0ff,line-height:1.2
        classDef first fill-opacity:0
        classDef last fill:#bfb6fc
```

Resultado do conferir.py

Rode `python conferir.py` no fim e marque o status de cada etapa como aparece no resumo.

Etapa	Status	O que falta (se não completa)
0 Preparação	☑ completa	Nenhuma pendência (100% testado e aprovado)
1 Agente com tool	☑ completa	Nenhuma pendência (100% testado e aprovado)
2 Extração	☑ completa	Nenhuma pendência (100% testado e aprovado)
3 Guardrails	☑ completa	Nenhuma pendência (100% testado e aprovado)
4 APIs (pronta)	☑ completa	Nenhuma pendência (100% testado e aprovado)
5 MCP	☑ completa	Nenhuma pendência (100% testado e aprovado)
6 Especialistas	☑ completa	Nenhuma pendência (100% testado e aprovado)
7 Grafo	☑ completa	Nenhuma pendência (100% testado e aprovado)
8 Chefia	☑ completa	Nenhuma pendência (100% testado e aprovado)
9 Segurança	☑ completa	Nenhuma pendência (100% testado e aprovado)
10 Observabilidade	☑ completa	Nenhuma pendência (100% testado e aprovado)

Etapa	Status	O que falta (se não completa)
11 Entrega	☑ completa	Nenhuma pendência (100% testado e aprovado)

Reflexão final

Até 3 linhas cada.

1. Quem decide o caminho no seu sistema: o LLM, o grafo, as regras ou um humano? Dê um exemplo de cada.
   Resposta: Grafo: roteia o fluxo e paraleliza nós (ex: fan-out normas/escala). LLM: extrai entidades e redige despachos amigáveis. Regras: barram objetivamente violações de prazos/saldo N1-N7. Humano: chefia imediata que aprova ou rejeita via interrupt (N8).
2. Qual dado veio do MCP, qual de API externa e qual de tool local? Por que essa divisão?
   Resposta: MCP: dados corporativos transacionais protegidos (servidores e escala). API externa: serviços utilitários públicos (feriados e câmbio). Tool local: regras determinísticas da PCDF. Essa separação isola segurança, resiliência e velocidade.
3. Qual etapa foi a mais difícil, e quanto tempo levou?
   Resposta: A Etapa 7 (LangGraph com fan-out/in e ciclo de validação), demandando cerca de 45 minutos para alinhar os reducers do estado compartilhado, os loops condicionais de reescrita e os checkpoints persistentes em SQLite.